Basic (single-agent support bot design) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `single_agent_design.md`: a one-sentence request is where most real AI features actually start, and the easiest mistake to make is building far more than was asked for. Writing this design out on paper first — your stated assumptions, one clean agent, a clear handoff to a human — trains the real skill of sizing a design to match the request, not your ambitions. **If not:** your first real design conversation at work would be the first time you ever had to say out loud what you're deliberately *not* building, and why.

Write your own design first, in `practice/single_agent_design.md`.

The request: "answer questions about our product docs, hand off to a human if unsure."

Solution

Story: this keeps the same basic shape, but hardens it exactly where a real technical reviewer would push back: what "unsure" actually means in concrete terms, what information the human receives when it escalates, and what happens the day the search tool itself goes down. **If not:** the bot would confidently answer from loosely related, barely-relevant documents, and go completely silent the day search is unavailable.

Same shape, hardened at the two places a senior reviewer will actually push on:

- **Grounding-based "unsure" instead of self-reported confidence:** after retrieval, the prompt constrains the agent to answer only from the retrieved text, and to explicitly say "I don't know" if the retrieved text doesn't cover the question — not rely on the model's own stated confidence, which isn't reliably calibrated.
- **Handoff payload includes what was already tried:** the human doesn't start from zero — they see the question, what was searched, and *why* it triggered escalation. A score-based miss and a grounding-based miss mean different things: a score miss means the docs might not cover this topic at all; a grounding miss on a borderline retrieval might just mean the query needs rephrasing.
- **Explicit failure handling for the tool itself:** `search_docs` timing out or erroring is its own path (escalate with reason "docs search unavailable"), not silently swallowed or retried forever.

- **Escalation logging as a product signal:** every handoff is logged with its reason — this becomes a way to find gaps in the docs themselves (repeated escalations on the same topic means missing documentation, not just a bot problem).
- **Cost/latency named explicitly, and deliberately not solved yet:** one retrieval plus one generation call per question is cheap at the stated moderate volume. A caching layer is named as a future addition *if* volume grows, not built now — this is "requirements → architecture is matching, not inspiration" from Core Concepts, applied to what *not* to build yet.

Why each piece is here, not just "what production systems have": "If the score is low, hand off" sounds complete but isn't — it doesn't cover retrieval finding *something* plausible-looking that isn't actually a real answer, which is precisely how a support bot ends up confidently stating something false. A design with no answer for "what happens when your search tool itself goes down" is exactly the kind of gap a reviewer finds first — the failure-mode question Doc14 trains you to ask before anyone else does. Every piece here — the grounding check, the tool-failure path, escalation logging — traces back to a specific way the obvious design breaks, not to "what real production systems have."

Presenting this in an interview: Open with the shape — stated assumptions, one clean agent, no over-building — since that shows you can size a design to a one-sentence request. Then, before the interviewer even asks, volunteer the grounding-based unsure check and the tool-failure path. Naming those unprompted is what separates "can build a single-agent bot" from "can be trusted to own this in production," and it's exactly what this document's "Expected Behavior" section is asking you to do without being prompted.